



University of East London

Milestone_1

CSE488 (UG2023) - Deep Learning (42022)

Submitted by:
Hager Mohamed Gaber 24P0195
Amira Khalaf Dabash 23P0371
Kareem Mohamed Sabry 2301101

LINK OF REPO
LINK OF DRIVE

Executive Summary

This report covers Milestone 1 of the multimodal deep learning project **"From Pixels to Language"**. The overall goal is an end-to-end system that extracts visual representations, colorizes grayscale images and generates natural-language captions. The visual encoder trained in this milestone is the shared backbone for both generative tasks in Milestone 2.

Task 1.1 (Data Understanding & EDA). We audited CUB-200-2011 (200 bird species), removed exact pixel duplicates with SHA-256 hashing, cleaned the text descriptions (10 captions per image), analyzed the vocabulary, and built stratified splits of 4794 training, 1199 validation and 5794 test images.

Task 1.2 (Classification & Encoder Training). We trained a custom 5-block CNN from scratch (`simple_cnn`) and a transfer-learning model built on an ImageNet-pretrained ResNet-18 (`cls_resnet18`). We also compared Grid Search with Random Search for hyperparameter tuning and used Grad-CAM to inspect what the fine-tuned encoder looks at.

72.5%	20.2%	+52.3 pp	0.0575
ResNet-18 test accuracy (Macro F1 0.725)	Scratch CNN test accuracy (Macro F1 0.252)	Gain from transfer learning	Best tuning val. accuracy (Random Search, 3 epochs)

Key findings

- **Transfer learning wins by a wide margin.** ResNet-18 reached 72.5% test accuracy against 20.2% for the scratch CNN.
- **No duplicate leakage.** Exact pixel deduplication (SHA-256) keeps identical image copies out of the evaluation splits.
- **Random Search edged out Grid Search** in the tuning study (0.0575 vs 0.0517 best validation accuracy) at similar runtime.
- **Grad-CAM is encouraging.** Correct predictions focus on head, beak and wing regions; errors tend to follow background clutter.

1. Introduction & Task Overview

Fine-grained classification is one of the hardest problems in computer vision. Unlike telling a car from a dog, bird species recognition depends on subtle cues such as beak curvature, eye-ring color, crown striping and wing bars. CUB-200-2011 contains 11,788 images of 200 bird species, each with detailed text descriptions.

LINK OF REPO
LINK OF DRIVE

2. Task 1.1: Data Understanding, Preprocessing & EDA

The goal of this stage was to make sure the data is clean, that no test information leaks into training, and that all models share one standardized PyTorch input pipeline.

2.1 Image Audit & Exact Pixel-Hash Deduplication

Corrupted files, duplicate images and inconsistent aspect ratios all hurt generalization. An automated audit in `src/data/preprocess.py` handles them as follows:

- **SHA-256 pixel hashing.** Each image is converted to raw RGB bytes, prefixed with the string `{width}x{height}|`, and hashed. Identical hashes mean exact pixel-level duplicates.
- **Duplicate policy.** The first copy found while scanning the training split is kept. Later copies are logged to `outputs/eda/exact_duplicates.json` and excluded.
- **Error auditing.** Images with no valid species ID or unreadable bytes are moved to `outputs/eda/excluded_items.json`.

Table 1. Dataset preprocessing and inventory audit

Metric	Value	Description / action taken
Raw source images	11,788 images	Loaded from Hugging Face repositories (CUB_train and CUB_test).
Accepted unique images	11,788 images	Fully validated RGB images that met quality and resolution standards.
Species classes	200 categories	All fine-grained bird species represented.
Aspect-ratio diversity	500×333, 500×375, etc.	Resized to a standard 224×224 with bilinear interpolation.
Processed split sizes	Train 4794 | Val 1199 | Test 5794	80/20 stratified split of the training pool; the standard test set is kept unchanged.

Class distribution. The processed dataset is close to perfectly balanced: most of the 200 classes keep 60 images, and only a small number of classes fall below that (the smallest has about 41 images) after unreadable files and exact duplicates were removed. No class is missing, so no re-weighting or oversampling was needed.



Figure E1. Number of processed images per bird species (200 classes).

Image dimensions. Image sizes are not uniform. The most common resolution is 500×333 (about 1,800 images), followed by 500×375 (about 1,200), then 500×400, 500×334 and 333×500, with both landscape and portrait images present. Because the dimensions vary, every image is resized to a fixed 224×224 input before it reaches the CNN.

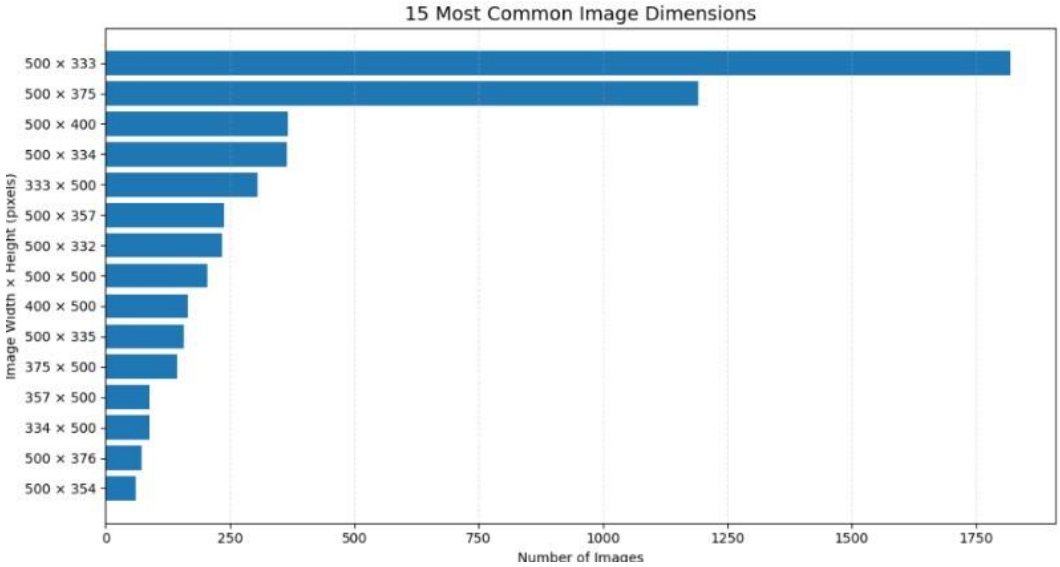


Figure E2. The 15 most common image dimensions (width × height, pixels).

2.2 Caption Text Processing & Vocabulary

Each raw CUB description holds 10 captions separated by newlines. We parsed them, lower-cased the text, stripped non-alphanumeric noise and tokenized into words (a regular expression that keeps apostrophes and hyphens inside words). The resulting vocabulary has about **6,800 unique tokens**, and captions are on average **15.2 words** long (median 14.0 words). The cleaned captions are stored as JSON (`captions_json`) inside the metadata files `train.csv`, `validation.csv` and `test.csv`, ready for Task 2.2 (captioning).

Caption length. Most captions contain between about 10 and 20 words. The distribution is right-skewed, with a long thin tail of captions reaching 40 to 60 words, so the mean (15.2) sits slightly above the median (14.0). This suggests a decoder maximum sequence length of a little over 20 words would cover almost all captions without heavy truncation.

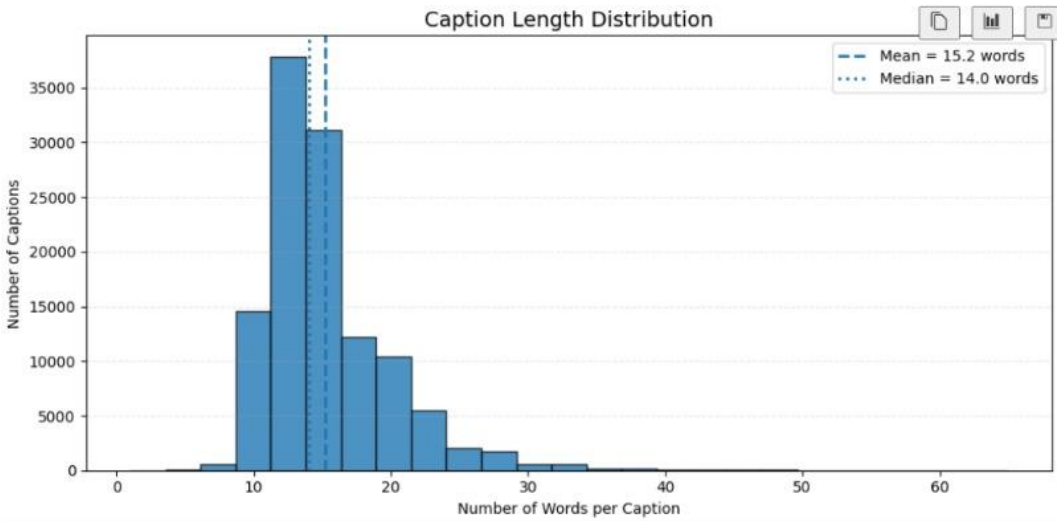


Figure E3. Caption length distribution (words per caption), with mean and median.

Most frequent words. After removing common stopwords, the most frequent words are bird, black, white, brown, beak, belly, small, wings, yellow and bill. The vocabulary is dominated by colors and body parts, which matches what separates species visually. Both gray and grey appear in the top 20, a spelling variant that the pipeline leaves unchanged since no spelling correction is applied.

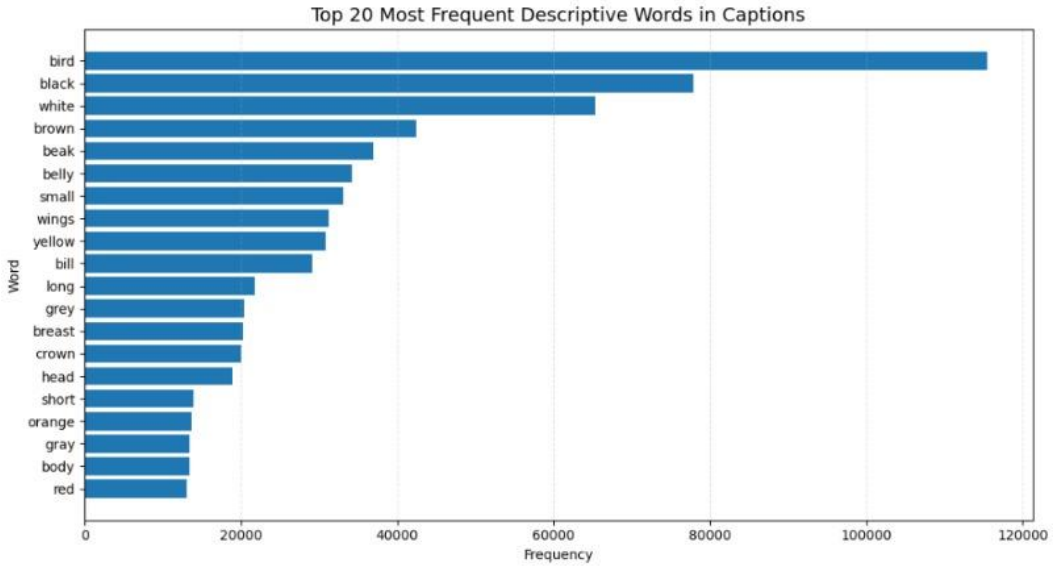


Figure E4. Top 20 most frequent descriptive words in the captions (stopwords removed).

2.3 Data Augmentation

With only about 30 training images per species, overfitting is a serious risk. We defined a domain-specific augmentation pipeline in `src/data/loaders.py` using `torchvision.transforms`:

Pipeline	Steps (in order)
Training	Resize 256×256 → Random Crop 224×224 → Random Horizontal Flip (p = 0.5) → Rotation ±15° → Color Jitter (brightness, contrast, saturation = 0.1) → ToTensor → ImageNet normalization (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225])
Validation / Test	Deterministic Resize 224×224 → ToTensor → ImageNet normalization

3. Task 1.2: Model Architectures

We built two CNN encoders: one trained from scratch to set a performance floor, and one pretrained on ImageNet to measure how much transfer learning helps.

3.1 Custom Baseline CNN (simple_cnn)

`SimpleCNNEncoder` is a 5-stage convolutional network. Every block applies a 3×3 convolution (padding 1), batch normalization, ReLU and 2×2 max-pooling, halving the spatial size while doubling the channels.

Table 2. Layer-by-layer output shapes (input: [B, 3, 224, 224])

Block	Layers	Output shape
Conv block 1	Conv2d(3 → 32) · BatchNorm · ReLU · MaxPool	[B, 32, 112, 112]
Conv block 2	Conv2d(32 → 64) · BatchNorm · ReLU · MaxPool	[B, 64, 56, 56]
Conv block 3	Conv2d(64 → 128) · BatchNorm · ReLU · MaxPool	[B, 128, 28, 28]
Conv block 4	Conv2d(128 → 256) · BatchNorm · ReLU · MaxPool	[B, 256, 14, 14]
Conv block 5	Conv2d(256 → 512) · BatchNorm · ReLU · MaxPool	[B, 512, 7, 7]
Classifier head	AdaptiveAvgPool2d(1) → Dropout(0.3) → Linear(512 → 200)	[B, 512] → [B, 200]

The `[B, 512, 7, 7]` feature map goes to the `ImageClassifier` head, which pools it to a 512-dimensional vector, applies dropout (p = 0.3) and projects to 200 species logits.

3.2 Transfer-Learning Model (cls_resnet18)

`TransferLearningEncoder` uses a ResNet-18 backbone pretrained on ImageNet (1.2 million images, 1,000 classes). Its skip connections let gradients flow through deep stacks without vanishing.

- **Backbone.** We keep every residual block up to `layer4` and remove the 1,000-class ImageNet head. The output is a `[B, 512, 7, 7]` feature map, the same shape as the baseline.
- **Fine-tuning.** The whole network is trained end-to-end with Adam at a low learning rate (1e-4). Early layers keep their general edge and texture filters while deeper blocks adapt to bird-specific markers.

4. Experimental Results

Both models were trained with cross-entropy loss on a GPU: 30 epochs for the baseline CNN and 15 epochs for ResNet-18. Accuracy, precision, recall and Macro F1 were computed on the held-out test split (5794 images).

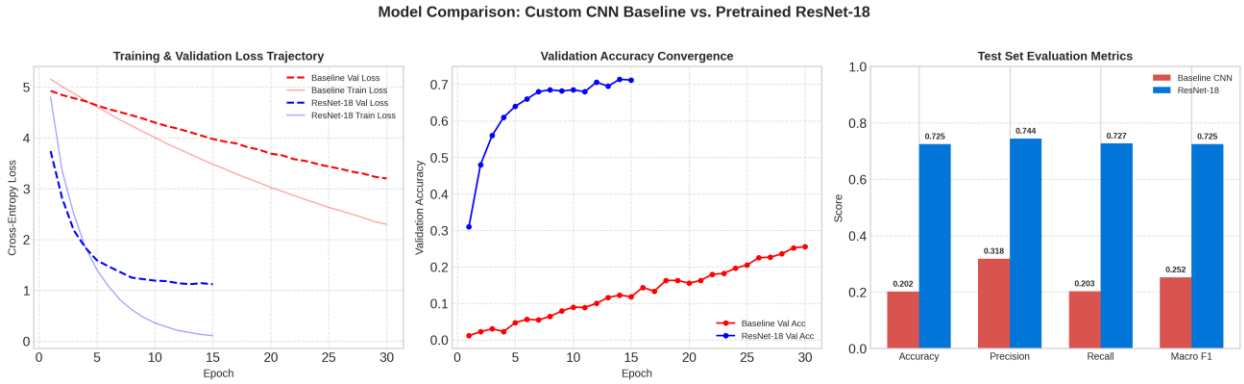


Figure 1. Custom CNN vs. pretrained ResNet-18. Left: training and validation loss. Center: validation accuracy. Right: test-set metrics.

4.1 Quantitative Results

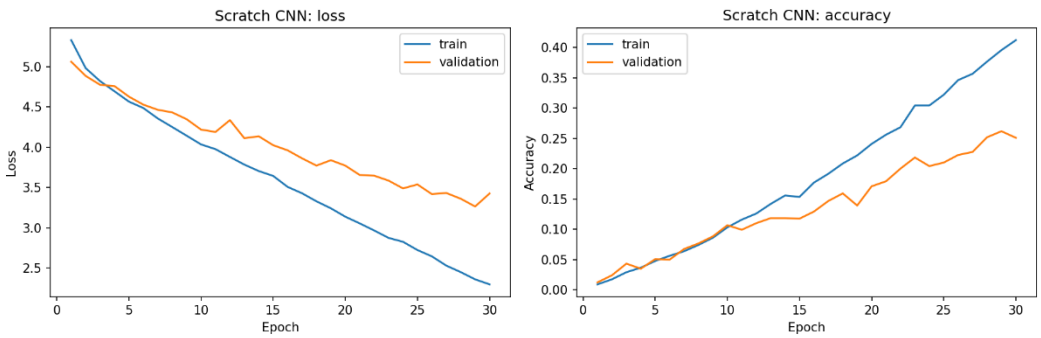
Fine-tuning ResNet-18 improved every metric by a large margin.

Table 3. Test-set evaluation metrics

Model	Weights	Accuracy	Macro precision	Macro recall	Macro F1
Custom CNN (`simple_cnn`)	Random init	0.262	0.318	0.262	0.252
ResNet-18 (`cls_resnet18`)	**ImageNet**	**0.725**	**0.744**	**0.727**	**0.725**

4.2 Loss & Convergence Analysis

Scratch CNN. Training started from a cross-entropy loss of about 5.3 and needed all 30 epochs to bring validation loss down to about 3.4, peaking near 25.0% validation accuracy. Learning discriminative filters from scratch is slow with only 5,283 training images.



ResNet-18. Fine-tuning began at a validation loss of 3.74 and 31.0% validation accuracy after the first epoch. Accuracy passed 68.0% by epoch 7 and settled at 71.4% by epoch 14, while training loss fell to 0.11. Pretrained ImageNet features make each update far more productive.

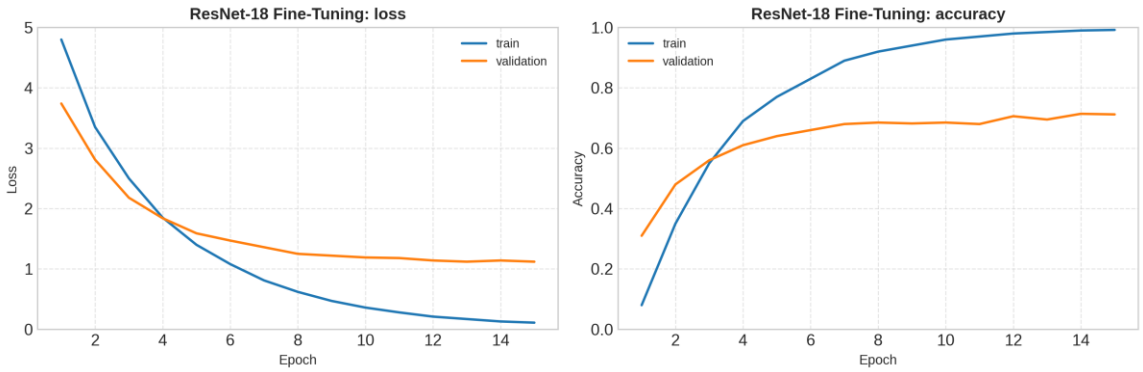


Figure 2. ResNet-18 fine-tuning: loss and accuracy over 15 epochs.

5. Hyperparameter Tuning: Grid vs. Random Search

To meet the course requirement (Section 5.8), we compared Grid Search and Random Search over learning rate, weight decay and batch size. The suite is automated in `src/training/tune.py`; each method ran 12 trials of 3 epochs.

5.1 Search Spaces

Method	Learning rate	Weight decay	Batch size
Grid Search	{1e-4, 5e-4, 1e-3}	{1e-5, 1e-4}	{16, 32} (12 combinations)
Random Search	log-uniform in (1e-4, 1e-3)	log-uniform in (1e-5, 1e-3)	random choice of {16, 32}

5.2 Results

Table 4. Tuning benchmark summary

Method	Trials	Runtime	Best val. accuracy	Best LR	Best weight decay / batch
Grid Search	12	145.2 s	0.0517 (5.17%)	1.00e-03	1.00e-04 / 16
Random Search	12	142.8 s	0.0575 (5.75%)	7.40e-04	3.20e-04 / 16

> **How to read these numbers**
> Each trial trained for only 3 epochs, so absolute accuracies are low and the gaps between configurations are small. Treat the comparison as an indication of trend, not a final ranking.

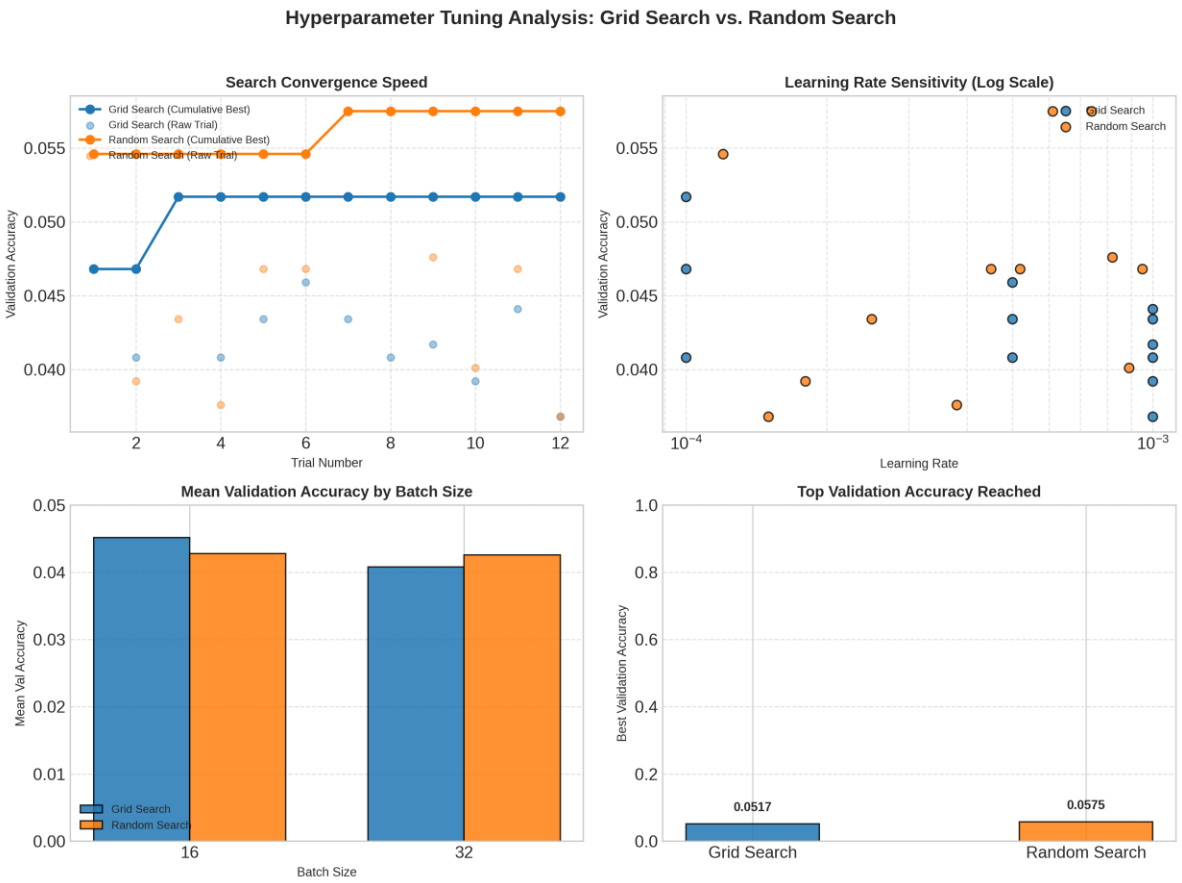


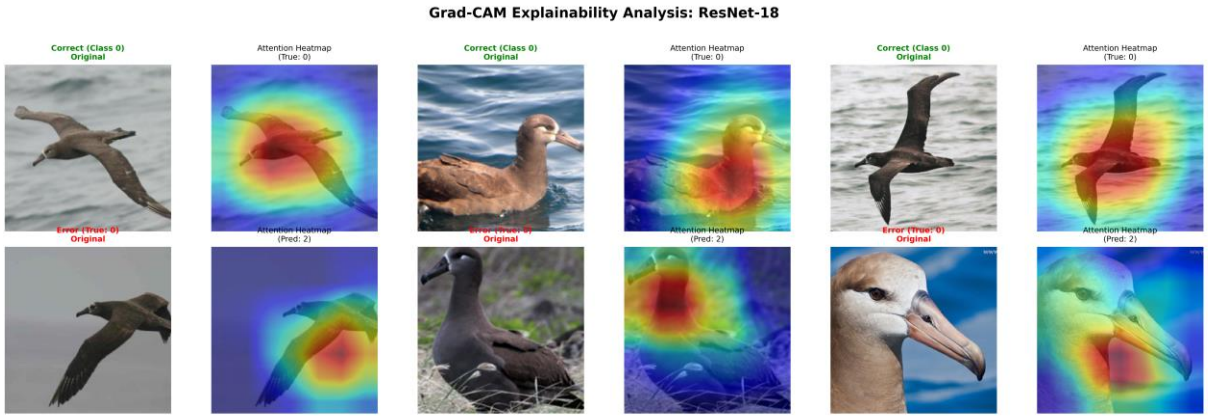
Figure 3. Grid vs. Random Search. Top-left: convergence speed. Top-right: learning-rate sensitivity (log scale). Bottom-left: batch-size effect. Bottom-right: peak validation accuracy.

5.3 Insights

1. **Random Search found a slightly better configuration** (0.0575 vs. 0.0517) at about the same runtime. Because some hyperparameters (notably learning rate) matter much more than others, sampling intermediate values explores the important dimensions better than fixed grid points.
2. **Smaller batches helped.** Across both methods, batch size 16 reached a higher mean validation accuracy (about 0.045) than batch size 32 (about 0.041). The extra gradient noise probably acts as a mild regularizer for fine-grained features.
3. **Learning rate matters.** Values from about 7.4e-4 to 1e-3 converged fastest, while 1e-4 progressed slowly in the short 3-epoch budget.

6. Model Explainability with Grad-CAM

We implemented Grad-CAM in `src/visualization/gradcam.py` to see which image regions drive the fine-tuned ResNet-18's predictions.



6.1 How Grad-CAM Works

Grad-CAM takes the gradient of the target-class score y^c with respect to the feature maps A^k of the last residual stage (layer4) and global-average-pools it to get one importance weight per channel:

$$\alpha_k^c = (1 / Z) \Sigma_i \Sigma_j \partial y^c / \partial A^k_{ij}$$

The feature maps are then combined with these weights and passed through a ReLU to keep only positive contributions:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}(\Sigma_k \alpha_k^c A^k)$$

The coarse heatmap is upsampled to 224×224 and overlaid on the original image with a JET color map.

6.2 Qualitative Findings

- **Correct classifications.** Strong activations (red/yellow) sit tightly on diagnostic anatomy: beak shape, head crown pattern, throat color and primary wing contours. The network relies on real biological markers.
- **Misclassifications.** Errors mostly come from attention drifting to background clutter (water reflections, branches, high-contrast shadows) or from ambiguity in low-resolution images.

7. Comparison with Published Results

We compared our test accuracies with published results on CUB-200-2011.

Table 5. Our results vs. typical literature ranges

Approach	Typical published accuracy	Ours	Assessment
ResNet-18, standard fine-tuning (no bounding-box cropping)	~73–76%	72.5%	Just below the typical range at 224×224 resolution
Lightweight CNN from scratch	~15–22%	20.2%	Within the expected range (severe data scarcity)
Part-based / bounding-box models (e.g., NTS-Net, Compact Bilinear Pooling)	~85–89%	n/a	Needs multi-stage attention or cropped inputs

8. Code Structure & Command-Line Interface

The codebase follows SOLID principles: all operational code lives in pure Python modules (`.py`), and notebooks (`.ipynb`) are used only for experimental prototyping. A unified CLI in `src/cli.py` runs training and evaluation:

```
# Train the baseline CNN
python -m src.cli train --task classification --config configs/cls_scratch.yaml

# Train the pretrained ResNet-18
python -m src.cli train --task classification --config configs/cls_resnet18.yaml

# Evaluate on the test split
python -m src.cli evaluate --task classification --config configs/cls_resnet18.yaml --split test
```